

Python Topic 14 — OOP: Classes & Objects

Full Stack Python + AI Roadmap • Taught in 3 layers: New to Python → Intermediate → Expert • Foundation for Pydantic, FastAPI, LangChain

Layer 1 — New to Python

A **class** is a blueprint for creating things. An **object** is an actual thing built from that blueprint. Class = cookie cutter, objects = the cookies.

```
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
        return f"{self.name} says Woof!"

# create objects from the blueprint
my_dog = Dog("Rex")
print(my_dog.bark())      # Rex says Woof!
print(my_dog.name)       # Rex
```

- `class Dog:` defines the blueprint
- `Dog("Rex")` creates an object (an "instance")
- `my_dog.name` / `my_dog.bark()` use the object's data and behavior

Layer 2 — Intermediate (real code + how it works)

`__init__` and `self`

```
class User:
    def __init__(self, name, email):
        self.name = name          # store data ON the object
        self.email = email

    def greet(self):
        return f"Hi, I'm {self.name}"

user = User("Akshay", "akshay@example.com")
```

- `__init__` = the constructor. Runs automatically on object creation; sets up initial data. (Like JS `constructor()`.)
- `self` = the object itself. Every method's first parameter is `self`; use it to access the object's own data (`self.name`).

JS comparison

```
// JS
class User {
  constructor(name, email) {
    this.name = name;
    this.email = email;
  }
  greet() { return `Hi, I'm ${this.name}`; }
}

# Python – self is explicit, written as first param of every method
class User:
  def __init__(self, name, email):
    self.name = name
    self.email = email
```

Python uses `self` (JS uses `this`), and `self` must be written **explicitly** as the first parameter — JS's `this` is implicit.

Instance attributes vs class attributes

```
class Dog:
    species = "Canis familiaris"    # class attribute – SHARED by all dogs

    def __init__(self, name):
        self.name = name            # instance attribute – unique per dog

rex = Dog("Rex")
milo = Dog("Milo")
rex.name        # "Rex"    (unique)
rex.species     # "Canis familiaris" (shared)
```

Class attributes are shared across all instances; instance attributes (set with `self.`) are unique to each object.

Dunder (Magic) Methods — Deep Dive

Dunder = "double underscore" (`__like_this__`). They let **your** object work with Python's built-in operations (`print`, `+`, `==`). Python calls them **automatically** — you never call them directly.

The problem they solve — without them, Python doesn't know how to handle your custom type:

```
a = Money(100)
print(a)          # 🤢 ugly!
print(a + b)      # ❌ TypeError: unsupported operand type(s) for +
a == Money(100)  # False 🤢 (even though amounts are equal!)
```

```
class Money:
    def __init__(self, amount):
        self.amount = amount

    def __str__(self):          # what print() shows (for users)
        return f"₹{self.amount}"

    def __repr__(self):        # developer/debug representation
        return f"Money({self.amount})"

    def __eq__(self, other):    # defines == (self=left, other=right)
        return self.amount == other.amount

    def __add__(self, other):   # defines + → returns a NEW object
        return Money(self.amount + other.amount)
```

```
a = Money(100)          # __init__ → self.amount = 100
b = Money(50)           # __init__ → self.amount = 50
print(a)                # Python calls a.__str__()          → ₹100
print(a + b)            # a.__add__(b) → Money(150), then __str__ → ₹150
print(a == Money(100))  # a.__eq__(Money(100))             → True
```

i The magic: you never call `__str__` / `__add__` / `__eq__` yourself. You use normal syntax (`print` , `+` , `==`) and Python routes to your dunder method.

You write	Python secretly calls	The dunder defines
<code>print(a)</code>	<code>a.__str__()</code>	how it displays (users)
<code>repr(a)</code> / REPL / in a list	<code>a.__repr__()</code>	debug display (developers)
<code>a + b</code>	<code>a.__add__(b)</code>	what <code>+</code> does
<code>a == b</code>	<code>a.__eq__(b)</code>	what <code>==</code> means
<code>len(a)</code>	<code>a.__len__()</code>	what <code>len()</code> returns
<code>a[0]</code>	<code>a.__getitem__(0)</code>	index access

i JS contrast: JS has `toString()` ($\approx$ `__str__`) but **can't** overload `+` or `==`. Python's dunder powers NumPy/Pandas/Pydantic doing things like `array1 + array2` or `df["col"] > 5`.

✓ **str vs repr:** `__str__` is for users (pretty); `__repr__` is for developers (precise, ideally recreates the object). If you define only one, define `__repr__`.



@property — Getters & Setters Deep Dive

`@property` lets a method be accessed like an attribute (no parentheses). The `@property` itself is the **getter**.

```
class Circle:
    def __init__(self, radius):
        self.radius = radius

    @property
    def area(self):
        # ← this IS the getter
        return 3.14159 * self.radius ** 2

c = Circle(5)
c.area          # 78.539... → reading works (getter runs), no parentheses
c.area = 100     # ✗ AttributeError – no setter, so area is READ-ONLY
```

Only a getter → `area` is read-only (it's computed from radius). To allow assignment with validation, add a **setter**:

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius        # the "real" stored value (underscore = internal)

    @property
    def celsius(self):
        # GETTER – runs when you READ .celsius
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        # SETTER – runs when you ASSIGN .celsius
        if value < -273.15:
            raise ValueError("Below absolute zero!")
        self._celsius = value
```

```
t = Temperature(25)
print(t.celsius)    # 25 → GETTER runs (reading)
t.celsius = 30      # → SETTER runs (assigning), validates, stores
t.celsius = -300    # ✗ ValueError: Below absolute zero! → setter caught it
```

Piece	Decorator	Runs when	Signature
Getter	<code>@property</code>	you read <code>t.celsius</code>	<code>def celsius(self)</code>
Setter	<code>@celsius.setter</code>	you assign <code>t.celsius = x</code>	<code>def celsius(self, value)</code>

i Key points: both methods share the **same name** (two halves of one property); the setter takes an extra **value** param (the right side of `=`); the real data lives in `_celsius` while `celsius` is the public gate that runs your code on read/write.

✓ JS parallel: same as JS `get` / `set` keywords — Python uses `@property` / `@name.setter` decorators. Both make a method masquerade as an attribute.

Layer 3 — More Expert Gotchas

1. The class attribute mutation trap (connects to Topic 1)

```
class Team:
    members = []          # 🚩 SHARED mutable class attribute!

    def add(self, name):
        self.members.append(name)

a = Team()
b = Team()
a.add("Akshay")
print(b.members)         # ['Akshay'] ← b sees it too! Shared list.
```

🚩 Fix: put mutable state in `__init__` so each instance gets its own — same trap as mutable default arguments.

```
class Team:
    def __init__(self):
        self.members = []    # unique per instance
```

2. `@classmethod` and `@staticmethod`

```
class User:
    count = 0

    def __init__(self, name):
        self.name = name
        User.count += 1

    @classmethod
    def from_dict(cls, data):    # alternative constructor
```

```

        return cls(data["name"])          # cls = the class itself

    @staticmethod
    def is_valid_email(email):              # utility, no self/cls needed
        return "@" in email

u = User.from_dict({"name": "Akshay"})    # class method as a factory
User.is_valid_email("a@b.com")            # True

```

- `@classmethod` receives the **class** (`cls`) — used for alternative constructors/factories
- `@staticmethod` receives **nothing** — just a utility grouped in the class

3. Private attributes (by convention)

```

class Account:
    def __init__(self):
        self.public = "anyone"
        self._internal = "please don't touch"    # single _ = "internal" convention
        self.__private = "name-mangled"         # double __ = name mangling

```

Python has **no true private** — it's convention. Single `_x` = "internal, don't use externally." Double `__x` triggers name-mangling (becomes `_ClassName__x`). Unlike JS's `#private`, these aren't enforced — Python trusts you.

4. `__slots__` — memory optimization (advanced)

```

class Point:
    __slots__ = ["x", "y"]    # restricts attributes, saves memory
    def __init__(self, x, y):
        self.x, self.y = x, y

```

For classes with millions of instances, `__slots__` skips the per-object `__dict__`, saving significant memory. Niche but good to know.

Interview Q&A Cards

Q: What are `__init__` and `self` ?

A: `__init__` is the constructor, run automatically on object creation to set initial state. `self` is the instance itself, passed as the first parameter of every method so it can access its own data.

Q: What's a dunder method?

A: A double-underscore method (`__str__` , `__eq__` , `__add__`) that Python calls automatically for built-in operations. It lets your objects work with `print` , operators, `len()` , indexing, etc.

Q: What does `@property` do, and how do you add a setter?

A: `@property` makes a method readable like an attribute (the getter). Add `@<name>.setter` with a `value` parameter to control assignment (validation/conversion). No setter → read-only.

Q: Class attribute vs instance attribute?

A: Class attributes are shared by all instances; instance attributes (set via `self.` in `__init__`) are unique per object. Never use a mutable class attribute — it's shared and leaks state.

Q: `@classmethod` vs `@staticmethod` ?

A: `@classmethod` receives the class as `cls` (used for factory/alternative constructors). `@staticmethod` receives neither `self` nor `cls` — a plain utility grouped in the class.

 **Memory hook:** Class = blueprint, object = instance. `__init__` builds it, `self` is the object referring to itself. Dunders let objects speak "Python." `@property` = getter, `@name.setter` = setter. Mutable state in `__init__` , never as a class attribute.

✓ **One-liner for interviews:** "A class is a blueprint; objects are instances. `__init__` initializes instance state via `self` . Class attributes are shared; instance attributes are per-object. Dunder methods like `__str__` / `__eq__` / `__add__` integrate objects with built-ins and operators. Use `@property` for computed attributes, `@classmethod` for factories, `@staticmethod` for utilities."